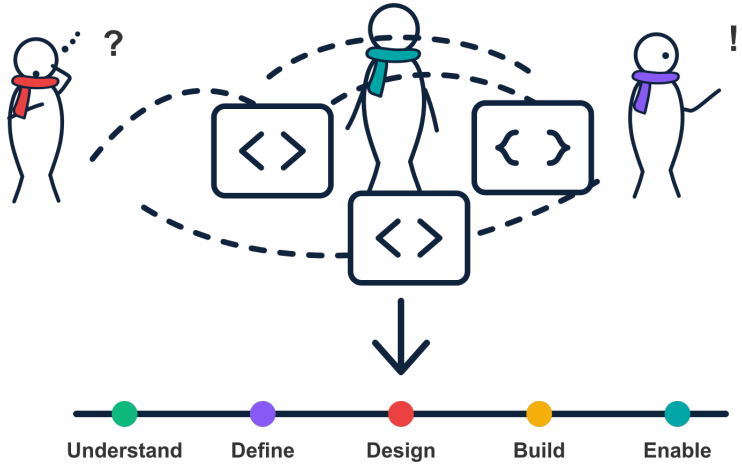
APIOps Cycles

Product strategy, architecture and delivery with AI

An introduction grounded in a financial-services case



Case study: APIsFirst with partner Osaango

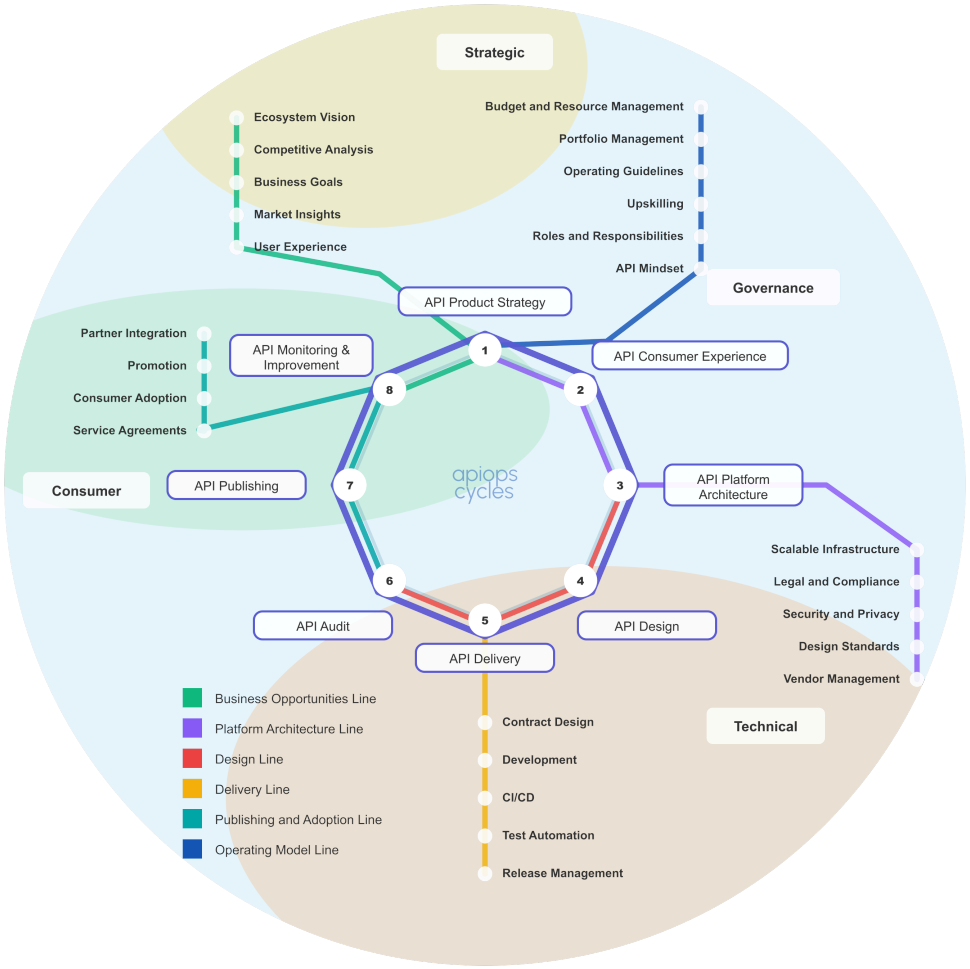
APIOps Cycles connects decisions across the lifecycle

An open method for connecting consumer value with architecture, design, delivery and continuous improvement.

Cycle view	Primary focus
Capability Productization	Define a reusable capability and the service promise.
API Productization	Develop, publish and improve an API as a product.
Integration Productization	Connect providers and consumers through reusable integrations.
Automation	Design and govern workflows, agents and operational controls.

This introduction follows API Productization, with a financial-services case and an illustrative payment continuation.

API Productization Cycle metro map



One lifecycle connects the work

1. API Product Strategy
2. API Consumer Experience
3. API Platform Architecture
4. API Design
5. API Delivery
6. API Audit
7. API Publishing
8. API Monitoring & Improvement

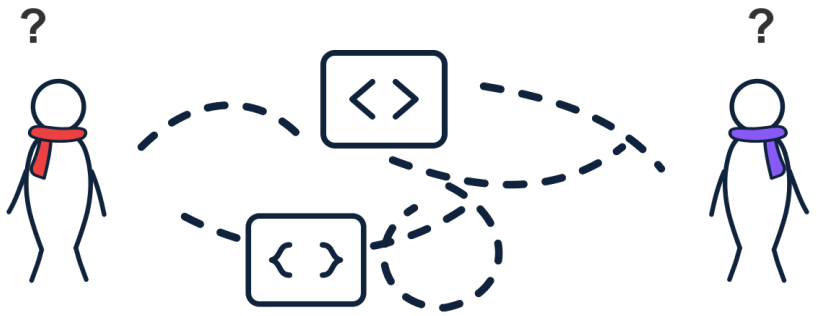
The engagement connected discovery to design, delivery and testing across these stations.

The challenge: dependencies across teams

Established API engineering practices

API products still depended on business and technology teams working across organisational boundaries.

Customer journeys and partner integrations had to fit internal services, legacy systems and compliance obligations.



Teams needed shared requirements, clearer ownership and coordinated delivery decisions.

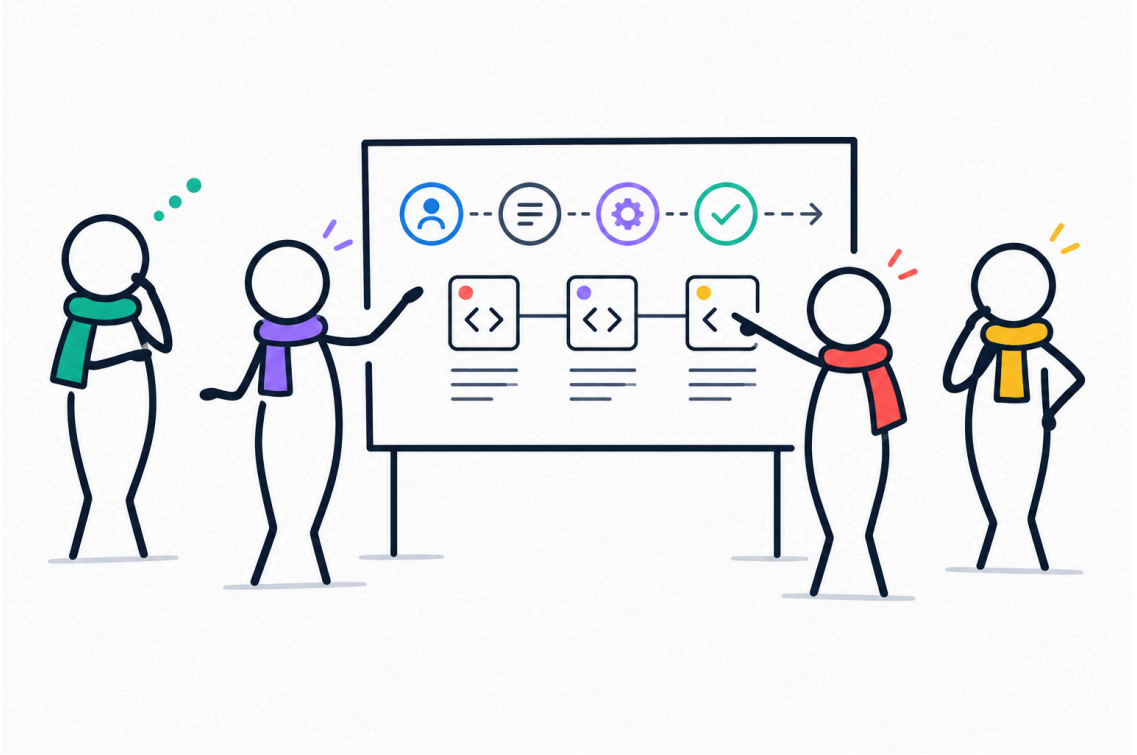
Collaborative discovery in the payments case

What the team did

APIsFirst and Osaango facilitated iterative workshops with business stakeholders, architects and engineers.

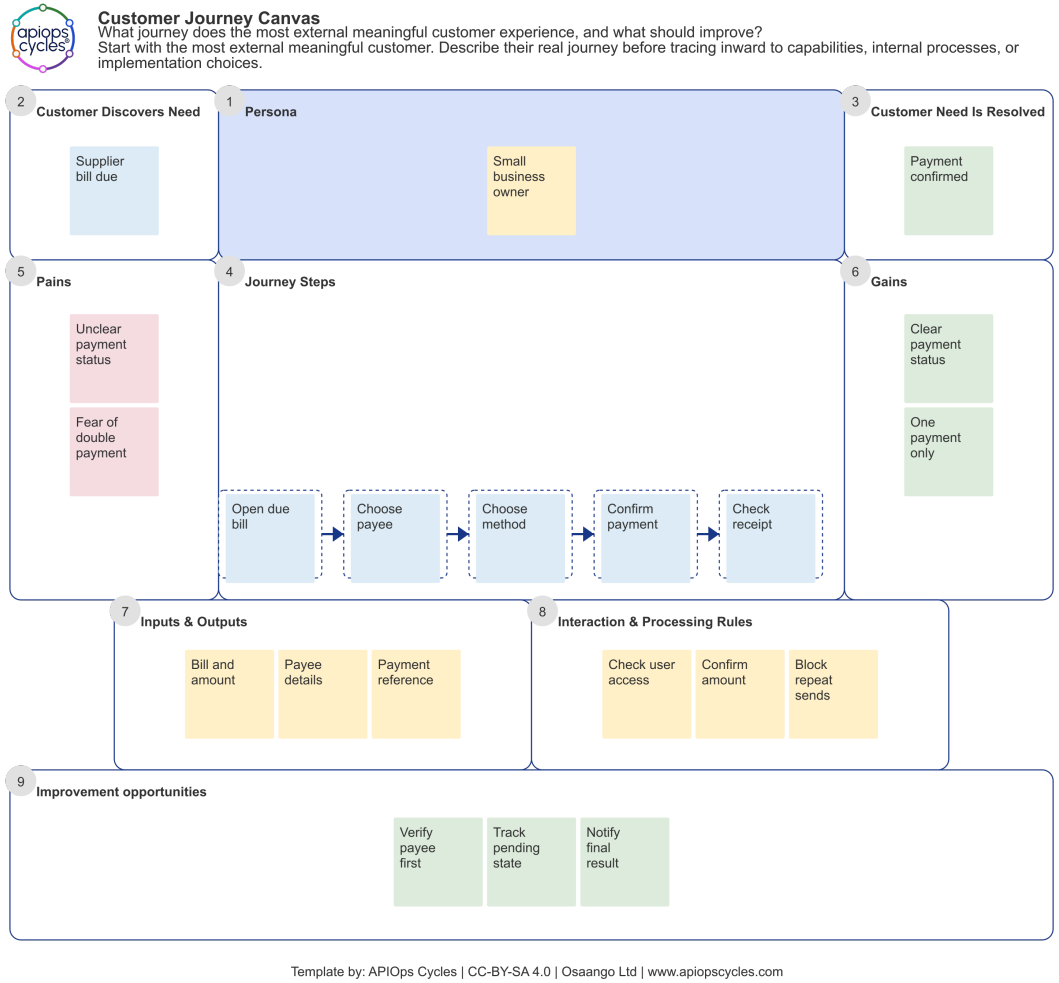
What the canvases made visible

Customer and API consumer journeys, value propositions, business rules and operating constraints.



Payments example: consumer details, payment method access, business risks and internal or partner channel needs.

Customer Journey Canvas: a supplier payment



Customer and trigger

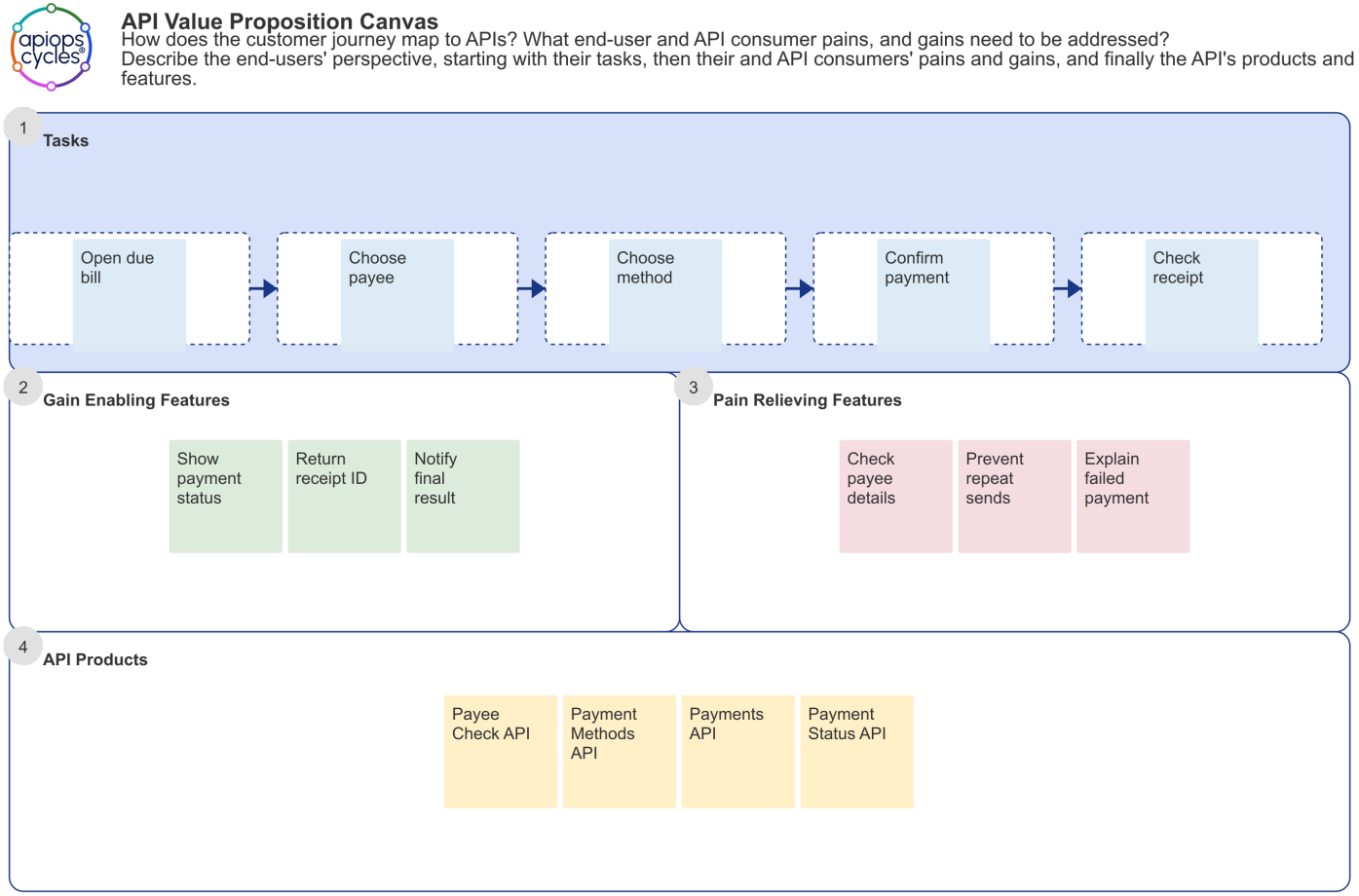
A small business owner has a supplier bill due and wants to know the payment has completed.

The customer journey

1. Open the due bill
2. Choose the payee
3. Choose a payment method
4. Confirm the payment
5. Check the receipt

Illustrative example. Not a client artefact.

API Value Proposition Canvas: the same journey



Customer concern to API feature

Unclear payment status: show progress and a receipt.

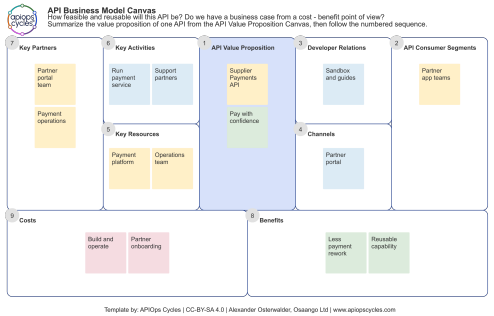
Repeat-payment concern: prevent duplicate submissions.

Candidate API products

- Payee Check API
- Payment Methods API
- Payments API
- Payment Status API

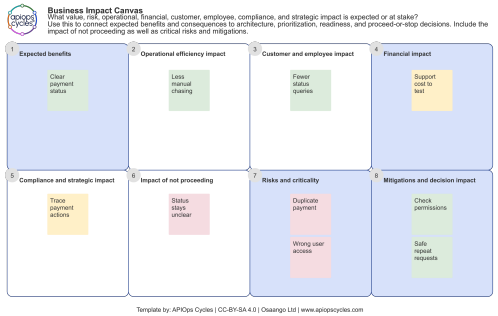
Illustrative example. API names are provisional.

Other canvases supporting API product decisions



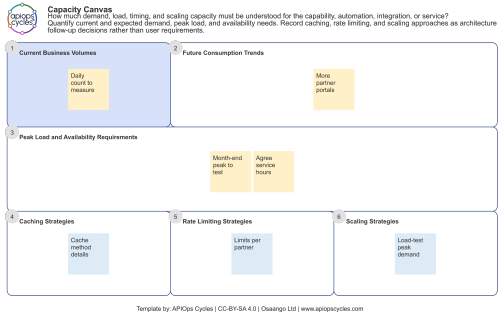
API Business Model Canvas

Value, partners and costs of the payments service



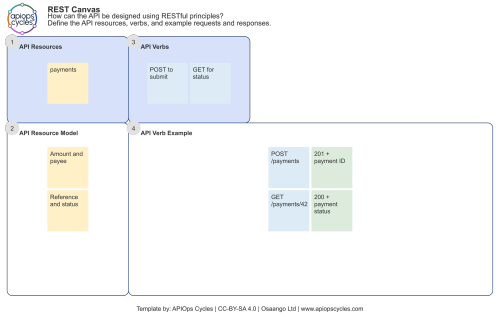
Business Impact Canvas

Expected benefits, consequences and critical risks



Capacity Canvas

Demand baseline, peak load and scaling decisions



REST API Canvas

Payment resources, verbs and response examples

Illustrative supplier-payment examples. Capacity measurements and assumptions require validation.

Discovery informed decisions across the lifecycle

Payment API design connected consumer needs to technical controls

Consumer tasks	Requirements	API design	Controls	Testing
Journeys and channel needs	Business rules and constraints	Domain model and contracts	Access, limits and monitoring	API test automation

Practical support

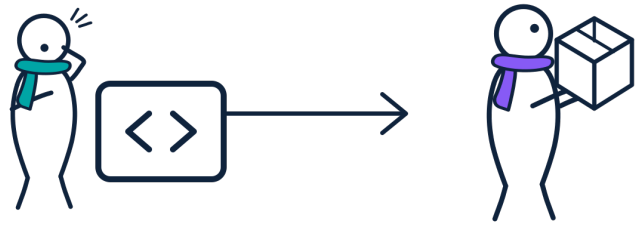
Guidance on API contracts and test automation. Cross-team input on unresolved architecture and security decisions.

AI modernisation guidance

Advice on using AI to convert legacy interfaces while retaining business requirements and constraints.

API product management in everyday work

- Consumers and value: Who uses the API, and what does it help them do?
- Ownership: Who is responsible for the product over its lifecycle?
- Reuse and adoption: How can other teams discover and use the capability?
- Lifecycle decisions: How should the product evolve as needs change?
- Governance and developer experience: How do practices support product outcomes?



Portfolio planning coordinated API releases

Which APIs to create, improve or release, and in what sequence



API portfolio planning

Priorities reflected partner integrations, UI development and dependencies between provider and consumer teams.

Capability improvement

A complementary view addressed API discoverability, governance, developer experience and product management.

The value across products, portfolio and ways of working

Connected work made decisions easier to discuss and repeat

Level	What the engagement connected	Qualitative value
API product	Consumer needs, business rules, contracts and technical controls	Shared understanding and more informed design
API portfolio	Provider and consumer dependencies with API priorities and releases	End-to-end visibility and coordinated delivery
Ways of working	Product ownership and lifecycle decisions with practical methods	Repeatable capability and continuous improvement

Qualitative benefits of the engagement. No quantified delivery or business impact is claimed.

Architecture: business needs become operating decisions

Method input	Decision it supports	Review with
Business Impact Canvas	Failure consequences, resilience and acceptable degradation	Product owner, security and operations
Locations Canvas	Trust boundaries, connectivity and data-location constraints	Architecture, security and compliance
Capacity Canvas	Demand, peaks, availability and scaling assumptions	Business and platform owners
Metrics and analytics	Observable service behaviour and consumer outcomes	Operations and API product owner

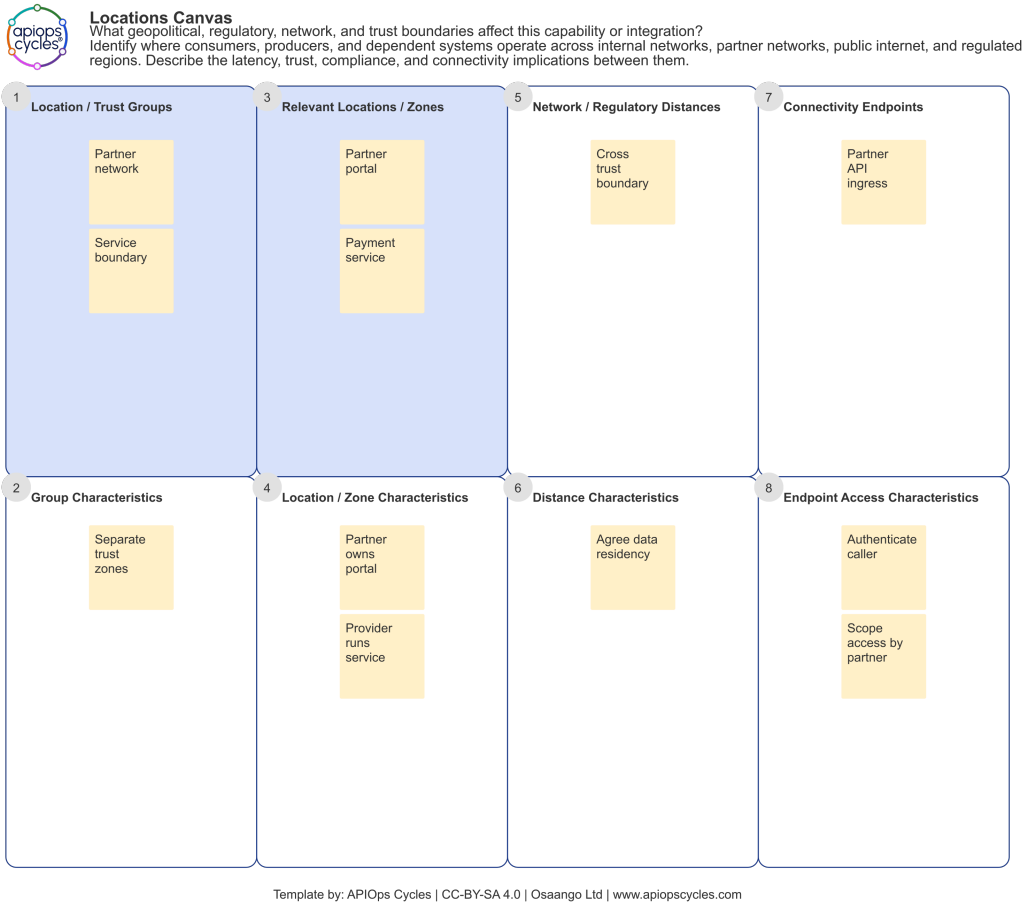
The next station needs architecture choices that stakeholders have reviewed.

A payment requirement can change the architecture

Customer or business need	Proposed design decision	Evidence to review
One payment per instruction	Define replay handling and a durable deduplication record	Repeated request and concurrent-submission tests
Clear progress and final outcome	Separate resource creation from completion of the business payment	State-transition and consumer walkthrough tests
A partner sees only its payments	Enforce access for the requested object and the caller context	Cross-partner denial tests
Dependable month-end service	Measure peak demand and agree capacity and recovery behaviour	Load tests and failure recovery evidence

Illustrative choices to validate, rather than facts about the client implementation.

Trust boundaries and capacity need explicit answers



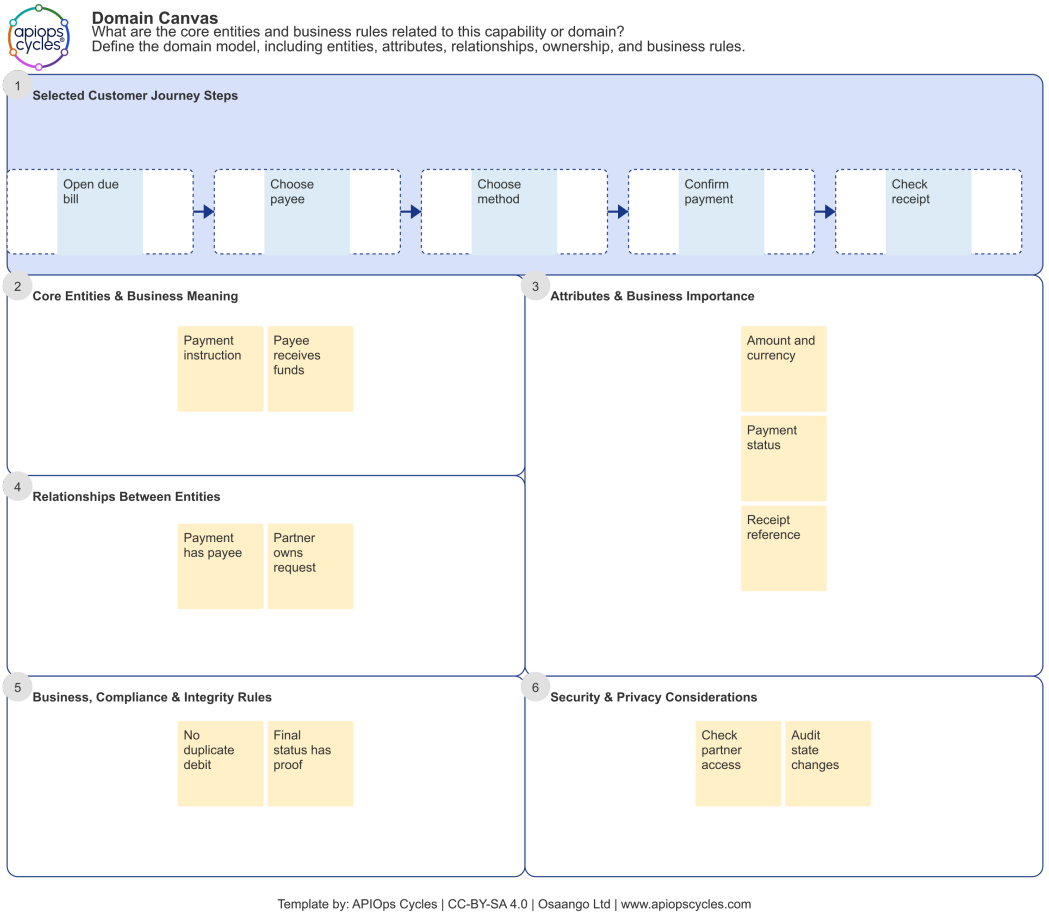
Trust and access

- Identify the caller and permitted payment objects.
- Agree exposed endpoints, network paths and data locations.

Demand and resilience

- Measure normal and peak demand before setting limits.
- Test retries, delays and recovery without duplicate processing.

API design starts with shared domain meaning



Payment and payee

A payment is an instruction with an amount, currency, payee and a lifecycle status.

Business rules shape the contract

- Keep accepted, pending and completed meanings distinct.
- Agree ownership and the source of truth for state changes.

Interaction design precedes the interface contract

Consumer task	Candidate interaction	Contract questions
Submit a payment instruction	REST request creating a payment resource	Required data, replay behaviour, creation response and errors
Check the outcome	REST query returning the current payment state	Status meaning, freshness and permitted access
Receive a final update	Event or callback if consumer needs justify it	Delivery, retry, ordering and duplicate handling
Compose a portal view	GraphQL only where its query model serves the need	Schema, authorisation and query-cost controls

The Interaction Canvas explores the flow; REST, Event or GraphQL canvases refine the chosen style.

Contract-first design makes behaviour reviewable

```
POST /payments
Idempotency-Key: <request-key>

201 Created
{
  "paymentId": "pay-42",
  "status": "pending"
}
```

Decisions around this operation

- Define amount, currency, payee and validation errors.
- Specify replay scope, retention and conflicting-payload behaviour.
- Declare authentication and required permissions.
- Keep resource creation distinct from payment completion.

Illustrative contract excerpt. The idempotency policy still needs agreement and implementation.

The scaffold gives people and AI a shared workspace

```
npm create apiops@latest payments
cd payments
npm run method:start
npm run method:canvases:new-api
```

Choose API Productization and REST in the prompts. The tooling requires Node.js 22 or later.

Project location	Purpose
specs/canvases/	Editable method decisions
specs/openapi/api.yaml	Project-owned API contract
docs/api/	Review and generated outputs
spectral/ and scripts/	Lint rules and audit checks
.github/workflows/	Automated review workflow

The starter includes a sample Catalog contract; your team builds the payment runtime in its chosen language.

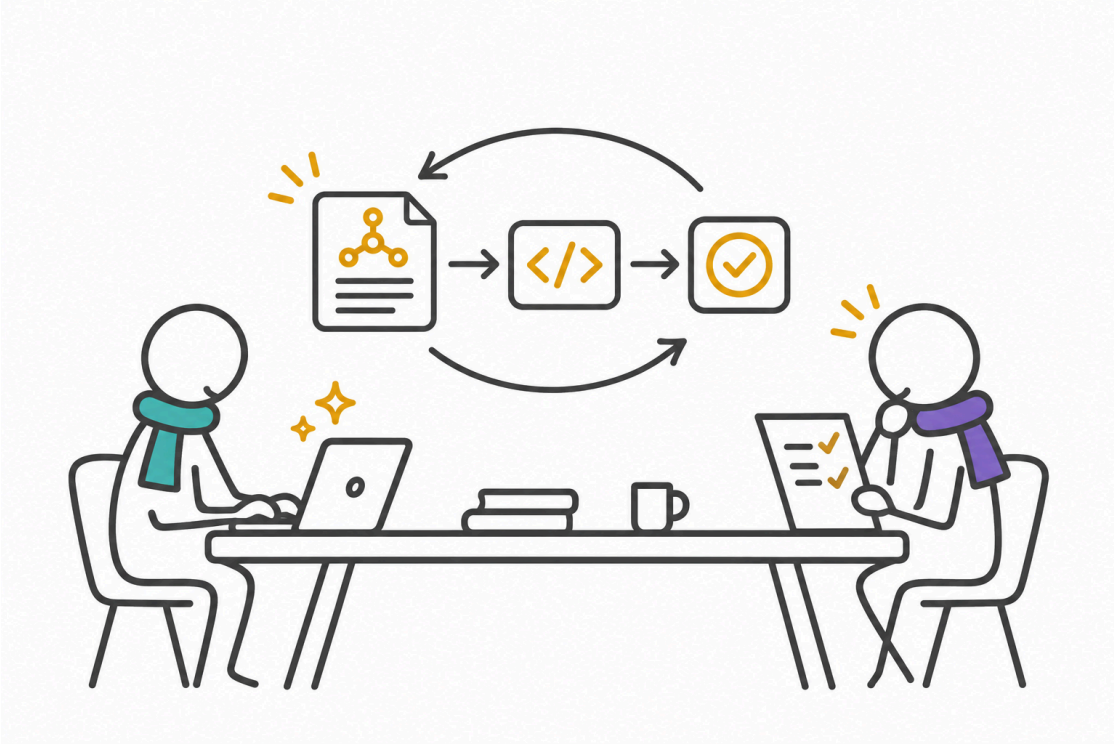
AI needs the decisions behind the contract

Input supplied to AI	What it constrains	Human owner
Journey and value proposition	Supported task, intended value and consumer expectations	Product and consumer representatives
Domain and architecture outputs	Meaning, ownership, trust and operational constraints	Domain, architecture and security owners
Validated contract and style guide	Interface behaviour, validation and shared conventions	API designer
Audit findings and test evidence	Known gaps, acceptance criteria and the next bounded change	Engineering and readiness owners

AGENTS.md and package skills provide a route through the method; the team still validates the decisions.

Building with AI works best as a bounded change

1. Select one reviewed operation and its acceptance criteria.
2. Ask for an implementation diff and tests against the contract.
3. Run checks, inspect behaviour and record unresolved decisions.
4. Have the responsible people review the evidence before merging.



Suggested task: implement payment submission with replay and cross-partner denial tests.

Tests turn the payment promises into evidence

Promise or risk	Example test	What to inspect
Contract consistency	Validate valid and invalid requests and responses	Schemas, error bodies and documented status codes
Replay protection	Repeat a request and submit concurrent duplicates	Stable result and no duplicate processing
Partner isolation	Read or alter a payment owned by another partner	Denied access and appropriate audit evidence
Clear failure behaviour	Simulate a slow or unavailable dependency	Agreed timeout, recovery and consumer-visible status
Peak demand	Exercise a measured peak traffic model	Latency, errors and resource consumption

These tests extend the scaffold with runtime evidence for the chosen implementation.

Linting checks the contract against explicit rules

```
npm run lint:openapi:full-crud
```

Scaffold profile	Rules in the inspected template	Use in this example
read-only	spectral:oas, a versioned server path and bans on write methods	For a genuinely read-only API
full-crud	The shared base rules, without the read-only verb bans	For the payment API with POST

The full-crud name permits writes; it does not require every CRUD operation.

The scaffold validate:openapi command runs both profiles, so a write-enabled API needs a deliberate profile choice.

API Audit brings evidence together across the lifecycle

```
npm run audit:design:full-crud
```

Coverage area	Evidence to review
Strategy and architecture	Consumer value, architecture decisions, trust boundaries and capacity
Design	Contract checks, examples, conventions and consumer review
Delivery	Functional, security, resilience and performance tests
Publishing	Documentation, access, support and readiness decisions
Improving	Runtime metrics, adoption and feedback connected to the next change

The report uses pass, partial, gap and not applicable. Manual items require evidence and owner review.

A check result needs a clear interpretation

Starter check or signal	What the inspected code checks	Evidence to add
Examples pass validation	Presence of an example somewhere in the specification	Actual example-to-schema validation
Validation workflow present	Existence of the expected workflow file	Successful execution and meaningful test results
Opaque identifier check	Names specific to the starter Catalog example	Checks adapted to payment identifiers and contracts
Manual readiness item	The checklist default status, commonly partial	Review of the actual evidence by the accountable owner

Adapt the starter checks to the API domain before treating coverage as a release decision.

CI can keep the review evidence current

Included scaffold workflow

1. Install dependencies with npm ci.
2. Lint the selected profile and generate audit reports.
3. Publish the JUnit coverage result as a GitHub check.

Extend for the payment implementation

- Run contract, integration, security and load tests.
- Define which lint findings, gaps and failed tests block release.
- Review changes to contracts, controls and generated evidence together.

Audit outputs: JSON for tooling, Markdown and HTML for review, JUnit for CI.

Publishing and improvement complete the product cycle

API Publishing

- Publish the reviewed contract and consumer-facing examples.
- Make access, onboarding and support ownership clear.
- Keep service terms and operational responsibilities visible.

API Monitoring & Improvement

- Observe payment completion, pending age, latency and errors.
- Combine runtime evidence with adoption and consumer feedback.
- Revisit architecture, contracts and priorities when assumptions change.

The next improvement starts with evidence from consumers and operations.

A first working cycle

1. Agree one API product, a consumer journey and an accountable owner.
2. Record the architecture choices and review one contract.
3. Build one path with AI assistance and meaningful tests.
4. Lint, audit and share findings before the next release decision.

Method and project resources

- apiopscycles.com
- apiops-cycles-method-data
- create-apiops
- CanvasCreator

Local reference versions checked: method-data 4.2.1 and create-apiops 1.1.4. 3 November 2026.